

Procedimiento de Ceremonia de Claves

Infraestructura de Clave Publica – Goya Ledger

Campo	Valor
OID del documento	1.3.6.1.4.1.99999.2.1.5
Version	1.0.0
Estado	Borrador
Fecha de vigencia	2024-01-01
CP gobernante	1.3.6.1.4.1.99999.2.1
CPS asociada	1.3.6.1.4.1.99999.2.2
Autoridad emisora	Goya Ledger CA
Jurisdiccion	Republica de Chile

Historial de revisiones

Version	Fecha	Autor	Descripcion
1.0.0	2024-01-01	Goya Ledger PKI Team	Procedimiento inicial alineado con DS 181 y ETSI EN 319 411-2

Tabla de contenidos

- [1. Objetivo y alcance](#)
- [2. Marco normativo](#)
- [3. Roles participantes](#)
- [4. Requisitos previos](#)
- [5. Procedimiento de generacion de clave raiz](#)
- [6. Procedimiento de generacion de CA intermedia](#)
- [7. Procedimiento de recuperacion de clave](#)
- [8. Procedimiento de destruccion de clave](#)
- [9. Registro y evidencia](#)

1. Objetivo y alcance

1.1 Objetivo

El presente documento establece el procedimiento operativo detallado para la generacion, custodia, recuperacion y destruccion de las claves criptograficas de la Autoridad Certificadora (CA) raiz y la CA intermedia de la Infraestructura de Clave Publica (PKI) de Goya Ledger.

El procedimiento tiene como finalidad garantizar que:

- La generacion de material criptografico se realice en un entorno controlado, auditable y verificable.
- La clave privada de la CA raiz jamas exista en texto claro fuera de un Modulo de Seguridad de Hardware (HSM) certificado.
- La cadena de custodia de cada fragmento de clave quede documentada con valor probatorio ante la normativa chilena.
- Todo el ciclo de vida de las claves cumpla con los requisitos de acreditacion del Decreto Supremo N.o 181 del Ministerio de Economia de Chile y las normas tecnicas europeas aplicables.

1.2 Alcance

Este procedimiento aplica a las siguientes operaciones criptograficas:

Operacion	CN del certificado	Algoritmo primario	Algoritmo PQC opcional
Generacion de CA raiz	Rust-BC Internal CA	ECDSA P-256	ML-DSA-65
Generacion de CA intermedia	Goya Ledger Intermediate CA	ECDSA P-256	ML-DSA-65
Recuperacion de clave raiz	Segun CN existente	Segun certificado	Segun certificado
Destruccion de clave	Segun CN existente	N/A	N/A

Los CN, algoritmos y periodos de validez estan definidos en el codigo fuente (src/pki.rs):

- INTERNAL_CA_CN = "Rust-BC Internal CA"
- INTERMEDIATE_CA_CN = "Goya Ledger Intermediate CA"
- CA raiz: validez de 10 anos (2024-01-01 a 2034-01-01)
- CA intermedia: validez de 5 anos desde la fecha de emision
- OID de politica de certificados: 1.3.6.1.4.1.99999.2.1
- URI de CPS: <https://goya.cl/pki/cp>

Fuera de alcance: La emision de certificados de nodo (operacion automatizada gestionada por `sign_node_cert()` y `provision_node_cert_if_absent()`), la gestion de claves de firma electronica simple (FES/Ed25519) y avanzada (FEA/ML-DSA-65) de usuarios finales, y la operacion rutinaria de la CA intermedia.

2. Marco normativo

2.1 Legislacion chilena

Norma	Relevancia
Ley N.o 19.799	Ley sobre Documentos Electronicos, Firma Electronica y Servicios de Certificacion. Establece el marco legal para prestadores de servicios de certificacion (PSC) acreditados.
Decreto Supremo N.o 181 (Ministerio de Economia, 2002, modificado 2019)	Reglamento de la Ley 19.799. Articulo 13: requisitos de seguridad para generacion y custodia de claves de CA. Articulo 14: obligacion de ceremonia formal con testigos y acta notarial. Articulo 17: requisitos de disponibilidad y respaldo de claves.
NCh-ISO 27001:2013	Sistema de Gestion de Seguridad de la Informacion. Referencia para controles de acceso fisico y logico al entorno de ceremonia.

2.2 Normativa europea (ETSI)

Norma	Seccion relevante	Requisito
ETSI EN 319 411-1	6.5.1	Requisitos generales de gestion del ciclo de vida de claves de CA.
ETSI EN 319 411-2	6.5.1 – 6.5.5	Requisitos reforzados para CA que emiten certificados cualificados: generacion en dispositivo seguro de creacion de firma cualificada (QSCD), division de clave con esquema M-de-N, presencia de

Norma	Seccion relevante	Requisito
ETSI EN 319 401	7.5	testigos cualificados.
		Politica de seguridad del prestador de servicios de confianza (TSP). Gestion de claves criptograficas.
ETSI TS 119 312	Tabla 1	Algoritmos criptograficos y longitudes de clave recomendadas.
		ECDSA P-256 aceptado hasta 2030. ML-DSA-65 como preparacion post-cuantica.

2.3 Estándares de seguridad criptografica

Estandar	Aplicacion
FIPS 140-3 (Level 2 minimo, Level 3 recomendado)	Certificacion requerida para el HSM utilizado en la generacion y almacenamiento de claves de CA. El HSM debe contar con certificacion vigente emitida por el CMVP (Cryptographic Module Validation Program) del NIST.
FIPS 186-5	Estandar de firma digital. Define ECDSA sobre curvas P-256, P-384, P-521.
NIST SP 800-57 Part 1	Recomendaciones de gestion del ciclo de vida de claves criptograficas. Periodos de vigencia, transiciones de algoritmo, destruccion segura.
NIST FIPS 204	Estandar ML-DSA (Module-Lattice-Based Digital Signature Algorithm). Define ML-DSA-44, ML-DSA-65 y ML-DSA-87. Goya Ledger utiliza ML-DSA-65 como algoritmo PQC.

2.4 Correspondencia con el codigo fuente

La implementacion de la ceremonia en Goya Ledger se encuentra en `src/pki_ceremony.rs`. Los pasos obligatorios validados programaticamente son:

<code>CeremonyStep::EnvironmentCheck</code>	-> Seccion 5.1 de este documento
<code>CeremonyStep::KeyGeneration</code>	-> Seccion 5.3
<code>CeremonyStep::WitnessAttestation</code>	-> Seccion 5.8
<code>CeremonyStep::KeyVerification</code>	-> Seccion 5.8
<code>CeremonyStep::Activation</code>	-> Seccion 5.9

La configuracion por defecto (`CeremonyConfig::default()`) establece:

- `threshold = 2` (minimo de fragmentos para reconstruir la clave)
- `total_shares = 3` (total de fragmentos Shamir)
- `notary_required = true`
- `min_witnesses = 2`

3. Roles participantes

3.1 Definicion de roles

Los roles estan definidos en `src/pki_ceremony.rs` como el enum `CeremonyRole`:

Rol (<code>CeremonyRole</code>)	Cargo	Responsabilidades	Requisitos de identidad
Administrator	Administrador de CA	Dirige la ceremonia. Ejecuta los comandos en el HSM. Genera y firma el certificado raiz. Responsable de la integridad del procedimiento.	Cedula de identidad chilena vigente. Contrato de trabajo o mandato que acredite su designacion como administrador de la CA.
Custodian	Custodio de Claves (3 personas)	Recibe, custodia y protege un fragmento (share) de la clave privada de la CA raiz. Debe presentarse fisicamente para operaciones de recuperacion.	Cedula de identidad chilena vigente. Declaracion jurada de responsabilidad de custodia. Domicilio verificado.
Witness	Testigo (minimo 2 personas)	Observa y atestigua la correcta ejecucion de cada paso del procedimiento. Firma el acta de ceremonia. No tiene acceso al material criptografico.	Cedula de identidad chilena vigente. Independencia funcional respecto del administrador y los custodios.
Notary	Notario Publico	Da fe publica de la ceremonia. Certifica	Nombramiento vigente como

Rol (CeremonyRole)	Cargo	Responsabilidades	Requisitos de identidad
Auditor	Auditor Interno	la identidad de los participantes. Protocoliza el acta de ceremonia. Su presencia es obligatoria (notary_required = true).	notario publico en Chile conforme al COT.
		Verifica el cumplimiento de cada paso contra este procedimiento. Registra desviaciones. Emite informe de conformidad.	Cedula de identidad chilena vigente. Independencia funcional. Competencia demostrable en auditoria de sistemas de informacion.

3.2 Incompatibilidades

- Una misma persona natural **no puede** desempeñar mas de un rol en la misma ceremonia.
- Los custodios **no pueden** ser subordinados directos entre si ni del administrador.
- El auditor **no puede** haber participado en el diseno o implementacion del sistema PKI objeto de la ceremonia.
- El notario debe ser externo a la organizacion.
- Minimo total de participantes: **8 personas** (1 administrador + 3 custodios + 2 testigos + 1 notario + 1 auditor).

3.3 Registro de participantes

Cada participante se registra como un CeremonyParticipant con los siguientes campos:

```
CeremonyParticipant {
    name: String,           // Nombre completo segun cedula de
                           // identidad
    role: CeremonyRole,     // Rol asignado
    did: Option<String>,    // DID Goya del participante (si
                           // aplica)
    organization: Option<String>, // Organizacion a la que
                           // pertenece
}
```

4. Requisitos previos

4.1 Sala segura

La ceremonia debe realizarse en una sala que cumpla los siguientes requisitos:

Requisito	Especificacion	Verificacion
Acceso fisico controlado	Puerta con cerradura electronica o mecanica. Registro de ingreso/egreso de toda persona.	Verificar bitacora de acceso el dia de la ceremonia.
Sin conectividad de red	La sala no debe tener conexion a redes LAN, WiFi ni celular durante la ceremonia.	Desconectar puntos de red. Verificar con escaner de RF. Activar jaula de Faraday si esta disponible.
Sin camaras de vigilancia con grabacion remota	Las camaras de la sala deben grabar localmente o estar desactivadas. No debe existir transmision de video fuera de la sala.	Verificar con el responsable de seguridad fisica.
Temperatura y humedad controladas	Temperatura entre 18-C y 24-C. Humedad relativa entre 40% y 60%.	Registrar lectura al inicio de la ceremonia.
Mesa de trabajo limpia	Superficie despejada, sin dispositivos electronicos no autorizados.	Inspeccion visual previa.
Iluminacion adecuada	Suficiente para lectura de documentos y pantallas.	Verificacion visual.

4.2 Equipamiento de hardware

Equipo	Especificacion	Cantidad
HSM (Hardware Security Module)	FIPS 140-3 Level 2 minimo (Level 3 recomendado). Soporte para ECDSA P-256 y, opcionalmente, ML-DSA-65. Interfaz PKCS#11. Ejemplos: Thales Luna Network HSM 7, Utimaco SecurityServer, YubiHSM 2 (para entornos de menor escala).	1 unidad principal + 1 unidad de respaldo (opcional)
Estacion de trabajo air-gapped	Computador portatil o de escritorio sin disco duro interno, con arranque	1 unidad

Equipo	Especificacion	Cantidad
Medio de arranque	desde medio vivo (USB). Sistema operativo Linux minimalista (e.g., Tails, Ubuntu Server minimal). Sin conexion de red de ninguna clase. USB booteable con sistema operativo Linux y herramientas preinstaladas: OpenSSL 3.x, pkcs11-tool, softhsm2-util, Rust toolchain (nightly, segun rust-toolchain.toml), binario compilado de rust-bc.	1 unidad (verificar hash SHA-256)
Medios de respaldo	Sobres de seguridad con sello numerado inviolable (tamper-evident). Para almacenar los fragmentos Shamir de clave.	3 sobres (uno por custodio)
Medios USB para exportacion	USB cifrada con hardware (e.g., IronKey) para exportar certificado de clave publica y acta.	2 unidades
Impresora	Impresora local (no en red) para imprimir el acta de ceremonia.	1 unidad
Camara fotografica	Para registro fotografico de la ceremonia. Sin conexion de red.	1 unidad

4.3 Documentacion previa

Antes de la ceremonia, el administrador debe tener disponibles los siguientes documentos:

1. **Este procedimiento** impreso, con firma de aprobacion del responsable de seguridad.
2. **Politica de Certificados (CP)** vigente – docs/policy/CP.md (OID 1.3.6.1.4.1.99999.2.1).
3. **Declaracion de Practicas de Certificacion (CPS)** vigente – docs/policy/CPS.md (OID 1.3.6.1.4.1.99999.2.2).
4. **Plan de Seguridad** – docs/policy/PLAN-SEGURIDAD.md.
5. **Plan de Contingencia** – docs/policy/PLAN-CONTINGENCIA.md.
6. **Lista de participantes** con nombre completo, RUN, rol asignado y organizacion.
7. **Checklist pre-ceremonia** completado (Anexo A).
8. **Plantillas de acta** impresas (Anexo B).

9. Plantillas de sobre de custodio impresas (Anexo C).

4.4 Software preinstalado en la estacion de trabajo

La estacion air-gapped debe contar con los siguientes binarios y bibliotecas verificados:

```
# Verificar la presencia de herramientas requeridas
openssl version          # Debe ser >= 3.0.0
pkcs11-tool --version    # Parte de OpenSC
softhsm2-util --version  # SoftHSM para pruebas (NO para
                          produccion)
rustc --version          # Nightly toolchain segun rust-
                          toolchain.toml
cargo --version          # Gestor de paquetes Rust

# Verificar el binario de Goya Ledger (compilado previamente en
                          entorno controlado)
sha256sum /opt/goya/rust-bc
# Comparar con el hash publicado en el repositorio de artefactos
                          firmado
```

4.5 Verificacion de integridad del HSM

Antes de la ceremonia, el administrador debe verificar:

```
# 1. Verificar el firmware del HSM (el comando varia segun
    fabricante)
# Ejemplo para Thales Luna:
lunacm -c hainfo

# 2. Verificar que el HSM este en estado de fabrica o que la
#     particion designada este vacia
pkcs11-tool --module /usr/lib/libCryptoki2_64.so --list-objects

# 3. Registrar el numero de serie del HSM
pkcs11-tool --module /usr/lib/libCryptoki2_64.so --list-slots
```

5. Procedimiento de generacion de clave raiz

5.0 Apertura de la ceremonia

Paso	Accion	Responsable	Verificacion
5.0.1	Registrar la hora de inicio (UTC) en el acta.	Administrador	Notario verifica la hora contra reloj independiente.
5.0.2	Verificar la identidad de cada participante	Notario	Auditor contrasta la lista de participantes con

Paso	Accion	Responsable	Verificacion
	mediante cedula de identidad vigente. Registrar nombre completo, RUN y rol en el acta.		la convocatoria previa.
5.0.3	Leer en voz alta el objetivo de la ceremonia y el procedimiento que se seguira.	Administrador	Testigos confirman que el procedimiento coincide con este documento.
5.0.4	Solicitar a cada participante que entregue dispositivos electronicos personales (telefonos, relojes inteligentes, tablets). Depositarlos en contenedor sellado fuera de la sala.	Administrador	Auditor verifica que ningun participante conserve dispositivos.
5.0.5	Cerrar la sala. A partir de este momento, no se permite el ingreso ni la salida de personas hasta que se complete la ceremonia o se declare su aborto.	Administrador	Testigos y notario confirman el cierre.

5.1 Verificacion de integridad del entorno (CeremonyStep::EnvironmentCheck)

Paso	Accion	Responsable	Verificacion
5.1.1	Verificar que la estacion de trabajo no tiene disco duro interno. Abrir la tapa inferior y mostrar la bahia vacia a los testigos.	Administrador	Testigos y auditor confirman visualmente. Registrar en acta.
5.1.2	Verificar la integridad del medio de arranque USB.	Administrador	Calcular sha256sum del medio y comparar con el hash de referencia

Paso	Accion	Responsable	Verificacion
5.1.3	Arrancar la estacion de trabajo desde el medio USB.	Administrador	publicado. Registrar ambos hashes en el acta. Verificar que el sistema arranca sin acceso a red: ip link show no debe mostrar interfaces activas (excepto lo).
5.1.4	Verificar la integridad del binario de Goya Ledger.	Administrador	sha256sum / opt/goya/rust-bc y comparar con hash de referencia. Registrar en acta.
5.1.5	Verificar el estado del HSM.	Administrador	Ejecutar los comandos de la seccion 4.5. Confirmar que la particion esta vacia o en estado de fabrica. Registrar numero de serie del HSM en el acta.
5.1.6	Verificar que no existe conectividad de red.	Administrador	ping -c 1 8.8.8.8 debe fallar. nmcli device status debe mostrar todos los dispositivos como desconectados. rfkill list debe mostrar todas las interfaces inalambricas bloqueadas.
5.1.7	Registrar condiciones ambientales.	Auditor	Temperatura, humedad, hora UTC.
5.1.8	El auditor firma el paso EnvironmentCheck en el acta.	Auditor	Los testigos refrendan.

Registro programatico:

```
ceremony.record_step(
    CeremonyStep::EnvironmentCheck,
    "nombre_administrador",

    "Entorno verificado: air-gap confirmado, HSM S/N XXXX,
    SHA-256 binario: abcd1234...",
    timestamp_utc,
);
```

5.2 Inicializacion del HSM

Paso	Accion	Responsable	Verificacion
5.2.1	Inicializar la particion del HSM con un nuevo Security Officer (SO) PIN.	Administrador	El SO PIN debe tener al menos 8 caracteres alfanumericos. El administrador lo genera, lo memoriza y lo introduce directamente en el HSM sin mostrarlo a ningun participante.
5.2.2	Crear un User PIN para la particion.	Administrador	El User PIN debe tener al menos 8 caracteres. Se introduce directamente en el HSM.
5.2.3	Verificar la inicializacion.	Administrador	<code>pkcs11-tool --module \$HSM_LIB --list-slots</code> debe mostrar la particion inicializada.
5.2.4	Abrir una sesion PKCS#11 autenticada.	Administrador	<code>pkcs11-tool --module \$HSM_LIB --login --pin \$USER_PIN --list-objects</code> debe retornar una lista vacia.

Comandos de referencia (ajustar segun fabricante del HSM):

```
# Variables de entorno para la ceremonia (NO se almacenan en disco)
export HSM_PKCS11_LIB="/usr/lib/libCryptoki2_64.so"
```

```

export HSM_SLOT_ID=0

# Inicializacion (SoftHSM para entorno de prueba; usar el CLI del
# fabricante para HSM de produccion)
softhsm2-util --init-token --slot 0 --label "GoyaRootCA" \
    --so-pin "$SO_PIN" --pin "$USER_PIN"

# Verificacion
pkcs11-tool --module "$HSM_PKCS11_LIB" --list-slots
pkcs11-tool --module "$HSM_PKCS11_LIB" --login --pin "$USER_PIN" \
    --list-objects --slot "$HSM_SLOT_ID"

```

5.3 Generacion del par de claves (CeremonyStep: :KeyGeneration)

5.3.1 Generacion de clave ECDSA P-256 (algoritmo primario)

Paso	Accion	Responsable	Verificacion
5.3.1.1	Generar el par de claves ECDSA P-256 dentro del HSM.	Administrador	La clave privada nunca abandona el HSM en texto claro.
5.3.1.2	Asignar la etiqueta "goya-root-ca-ecdsa-p256" al par de claves.	Administrador	Verificar con <code>pkcs11-tool --list-objects</code> .
5.3.1.3	Registrar el fingerprint (SHA-256 de la clave publica DER) en el acta.	Administrador	Los testigos leen y confirman el fingerprint en pantalla.

Comandos:

```

# Generar par de claves ECDSA P-256 en el HSM
pkcs11-tool --module "$HSM_PKCS11_LIB" --login --pin "$USER_PIN" \
    --keypairgen --key-type EC:prime256v1 \
    --label "goya-root-ca-ecdsa-p256" \
    --id 01

# Verificar que la clave se genero correctamente
pkcs11-tool --module "$HSM_PKCS11_LIB" --login --pin "$USER_PIN" \
    --list-objects --type pubkey

# Exportar clave publica para calcular fingerprint
pkcs11-tool --module "$HSM_PKCS11_LIB" --login --pin "$USER_PIN" \
    --read-object --type pubkey --label "goya-root-ca-ecdsa-p256" \
    -o /tmp/root-ca-pubkey.der

# Calcular fingerprint

```

```
openssl dgst -sha256 /tmp/root-ca-pubkey.der
# Registrar el hash resultante en el acta
```

5.3.2 Generacion de clave ML-DSA-65 (algoritmo PQC, opcional)

Si la organizacion ha decidido implementar proteccion post-cuantica, se genera un segundo par de claves:

Paso	Accion	Responsable	Verificacion
5.3.2.1	Verificar que el HSM soporta ML-DSA-65 (FIPS 204).	Administrador	Consultar documentacion del fabricante. Si no soporta ML-DSA-65, esta seccion se omite y se registra la omision en el acta.
5.3.2.2	Generar el par de claves ML-DSA-65 dentro del HSM.	Administrador	Etiqueta: "goya-root-ca-ml-dsa-65".
5.3.2.3	Registrar el fingerprint de la clave publica PQC en el acta.	Administrador	Tamano esperado de la firma: 3309 bytes (Vec <u>u8</u> >, no [u8; 64], segun la convencion de Goya Ledger).

Nota: El codigo en `src/pki.rs` utiliza `KeyPair::generate()` de la biblioteca `rcgen`, que por defecto genera ECDSA P-256. La integracion con ML-DSA-65 para certificados de CA requiere una extension de la biblioteca o la generacion directa via PKCS#11 y la construccion manual del certificado X.509. El algoritmo ML-DSA-65 se utiliza activamente en Goya Ledger para firma electronica avanzada (FEA) segun se documenta en `src/signature/`.

Registro programatico:

```
ceremony.set_key_info(
    "sha256:abcdef1234567890...", // fingerprint SHA-256 de la
    // clave publica
    "ECDSA-P256",                  // o "ML-DSA-65" para clave PQC
);

ceremony.record_step(
    CeremonyStep::KeyGeneration,
    "nombre_administrador",
    "Par de claves ECDSA P-256 generado en HSM S/N XXXX, label
    goya-root-ca-ecdsa-p256",
    timestamp_utc,
);
```

5.4 Exportacion de clave publica

Paso	Accion	Responsable	Verificacion
5.4.1	Exportar la clave publica del HSM en formato DER.	Administrador	pkcs11-tool --read-object --type pubkey --label "goya-root-ca-ecdsa-p256" -o root-ca-pubkey.der
5.4.2	Convertir a formato PEM.	Administrador	openssl ec -pubin -inform DER -in root-ca-pubkey.der -outform PEM -out root-ca-pubkey.pem
5.4.3	Calcular el hash SHA-256 de la clave publica PEM.	Administrador	sha256sum root-ca-pubkey.pem – registrar en el acta.
5.4.4	Copiar la clave publica PEM a dos medios USB cifrados.	Administrador	Cada USB se entrega a un testigo diferente para su custodia temporal.
5.4.5	Los testigos verifican que el hash del archivo en su USB coincide con el registrado en el acta.	Testigos	sha256sum /media/usb/root-ca-pubkey.pem

5.5 Division de clave privada – Shamir Secret Sharing (2-de-3) (CeremonyStep: :KeySplit)

La clave privada de la CA raiz se divide utilizando el esquema de secreto compartido de Shamir con los siguientes parametros, definidos en CeremonyConfig::default():

- **Umbral (threshold):** 2 fragmentos minimos para reconstruccion.
- **Total de fragmentos (total_shares):** 3.

Paso	Accion	Responsable	Verificacion
5.5.1	Extraer la clave privada del HSM en forma cifrada (wrapped key)	Administrador	La clave nunca aparece en texto claro. Segun HsmSigningProvider::backup_info(): metodo de respaldo es “HSM-to-HSM key

Paso	Accion	Responsable	Verificacion
	utilizando CKM_AES_KEY_WRAP o el mecanismo equivalente del fabricante.		wrapping (CKM_AES_KEY_WRAP) or Shamir M-of-N split”.
5.5.2	Aplicar el esquema Shamir 2-de-3 sobre la clave cifrada (wrapped).	Administrador	Utilizar una implementacion auditada de Shamir (e.g., rusty-secrets, sharks, o el mecanismo M-de-N nativo del HSM si lo soporta).
5.5.3	Verificar matematicamente que la recombinacion de 2 fragmentos cualesquiera reconstituye la clave original.	Administrador	Probar las 3 combinaciones posibles: {S1,S2}, {S1,S3}, {S2,S3}. Comparar hash del resultado con el hash de la clave wrapped original.
5.5.4	Imprimir cada fragmento en papel resistente al agua y a la decoloracion (o grabarlo en placa metalica).	Administrador	Cada fragmento se imprime en una pagina separada, identificada con: numero de fragmento (1/3, 2/3, 3/3), fecha de la ceremonia, fingerprint de la clave, y nombre del custodio asignado.
5.5.5	Borrar de forma segura toda copia temporal de los fragmentos en la estacion de trabajo.	Administrador	shred -vfz -n 10 /tmp/share_* seguido de sync.

Registro programatico:

```
ceremony.record_step(
    CeremonyStep::KeySplit,
    "nombre_administrador",
    "Clave dividida con Shamir 2-of-3. Verificacion: 3
    combinaciones exitosas.",
    timestamp_utc,
);
```

5.6 Distribucion de fragmentos a custodios (CeremonyStep::ShareDistribution)

Paso	Accion	Responsable	Verificacion
5.6.1	Introducir cada fragmento impreso en un sobre de seguridad con sello numerado	Administrador	El administrador NO lee el contenido del fragmento. Lo manipula boca abajo.

Paso	Accion	Responsable	Verificacion
5.6.2	inviolable (tamper-evident). Registrar el numero de sello de cada sobre en el acta, asociandolo al numero de fragmento y al custodio asignado.	Administrador	Notario verifica y registra la correspondencia.
5.6.3	Entregar personalmente cada sobre al custodio correspondiente.	Administrador	El custodio firma un recibo de entrega que incluye: fecha, hora, numero de fragmento, numero de sello del sobre, fingerprint de la clave.
5.6.4	Cada custodio verifica que el sello del sobre esta intacto y lo firma de puno y letra sobre el sello.	Custodios	Notario atestigua la firma.
5.6.5	Informar a cada custodio sus obligaciones: almacenar el sobre en caja fuerte o boveda bancaria, no abrirlo, reportar inmediatamente cualquier dano o intento de acceso no autorizado.	Administrador	Custodio firma declaracion de custodia (Anexo C).

Registro programatico:

```

ceremony.record_step(
    CeremonyStep::ShareDistribution,
    "nombre_administrador",
    "3 fragmentos distribuidos a custodios. Sellos: #001, #002, #003.",
    timestamp_utc,
);

```

5.7 Generacion del certificado autofirmado de CA raiz

Paso	Accion	Responsable	Verificacion
5.7.1	Generar la solicitud de certificado (CSR) con los siguientes atributos:	Administrador	Ver tabla de atributos abajo.
5.7.2	Firmar el certificado con la clave privada residente en el HSM.	Administrador	El certificado es autofirmado (issuer = subject).
5.7.3	Verificar las extensiones del certificado.	Administrador y auditor	Ver tabla de extensiones abajo.
5.7.4	Exportar el certificado en formato PEM.	Administrador	Registrar hash SHA-256 del PEM en el acta.

Atributos del certificado raiz:

Campo	Valor	Referencia en codigo
Common Name (CN)	Rust-BC Internal CA	pki::INTERNAL_CA_CN
Organization (O)	Goya Ledger	—
Country (C)	CL	—
Not Before	2024-01-01T00:00:00Z	pki::CA_NOT_BEFORE
Not After	2034-01-01T00:00:00Z	pki::CA_NOT_AFTER
Serial Number	Aleatorio, 128 bits	—
Signature Algorithm	ecdsa-with-SHA256 (OID 1.2.840.10045.4.3.2)	KeyPair::generate() en rcgen

Extensiones X.509 del certificado raiz:

Extension	Valor	Critica
basicConstraints	CA:TRUE, pathlen: sin restriccion	Si
keyUsage	keyCertSign, cRLSign	Si
subjectKeyIdentifier	SHA-1 de la clave publica	No
certificatePolicies	OID 1.3.6.1.4.1.99999.2.1, CPS URI https://goya.cl/pki/cp	No

El código en `src/pki.rs` utiliza `BasicConstraints::Unconstrained` para la CA raíz y agrega la extensión `certificatePolicies` mediante la función `certificate_policies_extension(CP_OID, "https://goya.cl/pki/cp")`.

Comando de verificación del certificado generado:

```
# Verificar el certificado
openssl x509 -in root-ca-cert.pem -text -noout

# Verificar campos criticos:
# - Issuer == Subject (autofirmado)
# - Basic Constraints: CA:TRUE
# - Key Usage: Certificate Sign, CRL Sign
# - Validity: 2024-01-01 a 2034-01-01
# - Signature Algorithm: ecdsa-with-SHA256
# - Certificate Policies: 1.3.6.1.4.1.99999.2.1

# Verificar la firma del certificado
openssl verify -CAfile root-ca-cert.pem root-ca-cert.pem
# Debe retornar: root-ca-cert.pem: OK
```

5.8 Verificación cruzada del certificado (`CeremonyStep::WitnessAttestation` y `CeremonyStep::KeyVerification`)

Paso	Acción	Responsable	Verificación
5.8.1	Mostrar en pantalla el contenido del certificado (<code>openssl x509 -text</code>).	Administrador	Los testigos leen y verifican: CN, fechas de validez, algoritmo, extensiones.
5.8.2	Calcular el fingerprint SHA-256 del certificado.	Administrador	<code>openssl x509 -in root-ca-cert.pem -fingerprint -sha256 -noout</code> . Dictar el fingerprint en voz alta.
5.8.3	Cada testigo registra el fingerprint en su propia copia del acta.	Testigos	Comparar las copias. Deben coincidir.
5.8.4	El auditor verifica independientemente el certificado.	Auditor	Ejecuta los mismos comandos de verificación en una segunda terminal o calcula manualmente el hash.

Paso	Accion	Responsable	Verificacion
5.8.5	Firmar el paso de atestiguamiento.	Testigos y auditor	Registrar en acta con firma manuscrita.

Registro programatico:

```
ceremony.record_step(
    CeremonyStep::WitnessAttestation,
    "nombre_testigo_1",
    "Certificado verificado. Fingerprint SHA-256: abcd1234...
    confirmado por 2 testigos.",
    timestamp_utc,
);

ceremony.record_step(
    CeremonyStep::KeyVerification,
    "nombre_auditor",
    "Verificacion independiente exitosa. openssl verify OK.
    Extensiones conformes a CP.",
    timestamp_utc,
);
```

5.9 Activacion de la CA raiz (CeremonyStep::Activation)

Paso	Accion	Responsable	Verificacion
5.9.1	Almacenar el certificado raiz PEM en el HSM asociado a la clave privada.	Administrador	pkcs11-tool --write-object root-ca-cert.der --type cert --label "goya-root-ca-ecdsa-p256"
5.9.2	Verificar que el certificado almacenado en el HSM es identico al exportado.	Administrador	Exportar nuevamente y comparar hashes.
5.9.3	Copiar el certificado raiz PEM a los medios USB cifrados para su publicacion posterior.	Administrador	Dos copias en dos USB independientes.
5.9.4	Declarar la CA raiz como activa.	Administrador	Registrar en acta: "La CA raiz con CN 'Rust-BC Internal CA' y fingerprint SHA-256 [hash]"

Paso	Accion	Responsable	Verificacion
			queda activada a las [hora UTC] del [fecha].”
5.9.5	El notario da fe de la activacion.	Notario	Firma del notario en el acta.

Registro programatico:

```
ceremony.record_step(
    CeremonyStep::Activation,
    "nombre_administrador",
    "CA raiz activada. Cert almacenado en HSM. 2 copias USB
    entregadas.",
    timestamp_utc,
);
```

5.10 Cierre de la ceremonia de clave raiz

Paso	Accion	Responsable	Verificacion
5.10.1	Borrar de forma segura toda informacion sensible de la estacion de trabajo.	Administrador	shred -vfz -n 10 /tmp/* y apagar la estacion. Como arranco desde USB sin disco, los datos se pierden al apagar.
5.10.2	Verificar que el HSM queda bloqueado (logged out).	Administrador	Cerrar la sesion PKCS#11.
5.10.3	Finalizar el registro de ceremonia.	Administrador	Ver seccion 9.
5.10.4	Todos los participantes firman el acta.	Todos	Ver seccion 9.
5.10.5	El notario protocoliza el acta.	Notario	El acta original queda en poder del notario. Se entregan copias autorizadas a la organizacion y al auditor.
5.10.6	Devolver los dispositivos electronicos personales a los participantes.	Administrador	Verificar contra la lista de entrega.
5.10.7		Administrador	

Paso	Accion	Responsable	Verificacion
	Abrir la sala y registrar la hora de finalizacion.		Registrar en la bitacora de acceso.

Finalizacion programatica:

```
let record: CeremonyRecord =
    ceremony.finalize(timestamp_utc_fin)?;
// record.status == CeremonyStatus::Completed
// record.record_hash == compute_record_hash(&record)
// Formato del hash: SHA-256 de "id|ceremony_type|fingerprint|
// algorithm|started_at|completed_at|participants.len|
// steps.len"

assert!(verify_record(&record));

// Serializar el registro para archivo permanente
let json = serde_json::to_string_pretty(&record)?;
```

6. Procedimiento de generacion de CA intermedia

La CA intermedia es la autoridad operativa que emite certificados a los nodos de la red. Su clave se genera en una ceremonia separada o como continuacion inmediata de la ceremonia de clave raiz.

6.1 Prerrequisitos adicionales

- La CA raiz debe estar activa y su certificado debe estar disponible.
- El HSM debe contener la clave privada de la CA raiz (o debe reconstruirse desde los fragmentos Shamir; ver seccion 7).
- Se requieren los mismos participantes que en la ceremonia de clave raiz, o participantes que cumplan los mismos requisitos.

6.2 Generacion del par de claves de CA intermedia

Paso	Accion	Responsable
6.2.1	Generar un nuevo par de claves ECDSA P-256 en el HSM con la etiqueta "goya-intermediate-ca-ecdsa-p256".	Administrador
6.2.2	Registrar el fingerprint de la clave publica en el acta.	Administrador

```
pkcs11-tool --module "$HSM_PKCS11_LIB" --login --pin "$USER_PIN" \
    --keypairgen --key-type EC:prime256v1 \
```

```
--label "goya-intermediate-ca-ecdsa-p256" \  
--id 02
```

6.3 Emision del certificado de CA intermedia

El certificado de CA intermedia se emite firmado por la CA raiz, con los siguientes atributos:

Campo	Valor	Referencia en codigo
Common Name (CN)	Goya Ledger Intermediate CA	pki::INTERMEDIATE_CA_CN
Organization (O)	Goya Ledger	—
Country (C)	CL	—
Not Before	Fecha actual de la ceremonia	OffsetDateTime::now_utc()
Not After	5 anos desde la emision	now + Duration::days(365 * 5)
Issuer	CN=Rust-BC Internal CA	Firmado por la CA raiz
Signature Algorithm	ecdsa-with-SHA256	—

Extensiones X.509 del certificado de CA intermedia:

Extension	Valor	Critica
basicConstraints	CA:TRUE, pathlen:0	Si
keyUsage	keyCertSign, cRLSign	Si
authorityKeyIdentifier	Identificador de la CA raiz	No
subjectKeyIdentifier	SHA-1 de la clave publica de la CA intermedia	No
certificatePolicies	OID 1.3.6.1.4.1.99999.2.1, CPS URI https:// goya.cl/pki/cp	No

El codigo en src/pki.rs utiliza BasicConstraints::Constrained(0) para la CA intermedia, lo que significa que la CA intermedia puede emitir certificados de entidad final pero no puede crear sub-CAs.

La funcion CaHierarchy::generate() en src/pki.rs implementa este flujo:

```
// Simplificacion del flujo en CaHierarchy::generate():  
// 1. Genera CA raiz (autofirmada,  
//    BasicConstraints::Unconstrained)  
// 2. Genera CA intermedia (firmada por raiz,  
//    BasicConstraints::Constrained(0))  
// 3. Retorna CaHierarchy { root, root_cert_pem, intermediate,  
//    intermediate_cert_pem }
```

6.4 Verificacion de la cadena de certificados

```
# Verificar el certificado de CA intermedia contra la CA raiz
openssl verify -CAfile root-ca-cert.pem intermediate-ca-cert.pem
# Debe retornar: intermediate-ca-cert.pem: OK

# Verificar la cadena completa (util para validar la funcion
  chain_pem())
cat intermediate-ca-cert.pem root-ca-cert.pem > chain.pem
openssl verify -CAfile root-ca-cert.pem -untrusted intermediate-
  ca-cert.pem chain.pem

# Verificar atributos del certificado intermedio
openssl x509 -in intermediate-ca-cert.pem -text -noout | grep -A
  2 "Basic Constraints"
# Debe mostrar: CA:TRUE, pathlen:0

openssl x509 -in intermediate-ca-cert.pem -text -noout | grep
  "Issuer"
# Debe mostrar: Issuer: CN = Rust-BC Internal CA
```

6.5 Almacenamiento y activacion

Paso	Accion	Responsable
6.5.1	Almacenar el certificado de CA intermedia en el HSM.	Administrador
6.5.2	Exportar el certificado de CA intermedia y la cadena completa (chain_pem()) a los medios USB cifrados.	Administrador
6.5.3	Configurar las variables de entorno para la operacion en produccion.	Administrador

```
# Variables de entorno para la operacion de la CA intermedia
# (definidas en docs/api/configuration-guide.md)
export TLS_CA_CERT_PATH="/etc/goya/pki/intermediate-ca-cert.pem"
export TLS_CA_KEY_PATH="/etc/goya/pki/intermediate-ca-key.pem"

# Verificar que NodeCaConfig::from_env() puede cargar la
  configuracion
cargo run --bin rust-bc -- --verify-pki
```

6.6 Prueba de emision de certificado de nodo

Como verificacion final, se emite un certificado de prueba para un nodo ficticio:


```
# Emision de certificado de prueba (usando la API programatica)
# La funcion sign_node_cert() genera un par ECDSA P-256 para el
    nodo,
# crea un certificado firmado por la CA intermedia con ttl_days
    especificado
```

```
cargo test --lib pki::tests -- --nocapture
```

```
# Verificacion manual del certificado de nodo emitido
openssl verify -CAfile chain.pem node-test-cert.pem
```

La funcion `sign_node_cert()` en `src/pki.rs`: - Genera un nuevo par de claves ECDSA P-256 para el nodo. - Establece CN y DNS SAN al `node_id` proporcionado. - Firma con la clave de la CA intermedia. - Retorna `IssuedNodeCert { cert_der, cert_pem, key_pem }`.

7. Procedimiento de recuperacion de clave (reunion de custodios)

7.1 Causales de recuperacion

La reconstruccion de la clave privada de la CA raiz solo procede en los siguientes casos:

1. **Renovacion del certificado raiz** antes de su vencimiento (2034-01-01).
2. **Emision de un nuevo certificado de CA intermedia** (cada 5 anos).
3. **Compromiso de la CA intermedia** que requiera revocacion y reemision.
4. **Migracion a nuevo HSM** por obsolescencia del hardware.
5. **Migracion algoritmica** (e.g., transicion de ECDSA P-256 a ML-DSA-65).

7.2 Convocatoria

Paso	Accion	Responsable
7.2.1	El administrador emite convocatoria formal por escrito a al menos 2 de los 3 custodios, indicando la causal, fecha, hora y lugar.	Administrador
7.2.2	La convocatoria debe enviarse con al menos 5 dias habiles de anticipacion.	Administrador
7.2.3	Se convoca tambien a los testigos, notario y auditor con los mismos requisitos de la ceremonia original.	Administrador
7.2.4	Se verifica que los custodios convocados no	Administrador

Paso	Accion	Responsable
	hayan reportado incidentes con sus sobres.	

7.3 Procedimiento de reconstruccion

Paso	Accion	Responsable	Verificacion
7.3.1	Ejecutar los pasos 5.0 (apertura) y 5.1 (verificacion de entorno).	Administrador	Mismos requisitos que la ceremonia original.
7.3.2	Verificar la identidad de los custodios presentes.	Notario	Cedula de identidad vigente. Comparar con el registro de la ceremonia original.
7.3.3	Cada custodio presenta su sobre sellado.	Custodios	Verificar que los sellos estan intactos y que los numeros de sello coinciden con el registro del acta original.
7.3.4	Los custodios abren sus sobres en presencia del notario y los testigos.	Custodios	El notario registra la apertura.
7.3.5	Los fragmentos se introducen en la estacion de trabajo air-gapped para la recombinacion Shamir.	Administrador	La estacion debe cumplir los mismos requisitos de la seccion 4.
7.3.6	Ejecutar la recombinacion Shamir (2-de-3).	Administrador	Verificar que el hash de la clave reconstruida coincide con el fingerprint registrado en el acta original.
7.3.7	Importar la clave reconstruida al HSM (nuevo o existente).	Administrador	<code>pkcs11-tool --write-object</code> o mecanismo de importacion del fabricante.
7.3.8		Administrador	

Paso	Accion	Responsable	Verificacion
	Verificar que la clave importada puede firmar y verificar correctamente.		Firmar un dato de prueba y verificar con la clave publica del certificado raiz existente.
7.3.9	Ejecutar la operacion que motivo la recuperacion (renovacion, emision de CA intermedia, etc.).	Administrador	Segun la causal indicada en 7.1.
7.3.10	Si se requiere nueva division Shamir (e.g., cambio de custodios), ejecutar los pasos 5.5 y 5.6 con los nuevos parametros.	Administrador	Destruir los fragmentos anteriores (seccion 8).
7.3.11	Borrar de forma segura todo material criptografico temporal.	Administrador	<code>shred -vfz -n 10</code> sobre todos los archivos temporales.
7.3.12	Cerrar la ceremonia segun el paso 5.10.	Administrador	Acta firmada por todos los participantes.

7.4 Recuperacion fallida

Si la recombinacion Shamir falla (fragmentos corruptos, hashes no coinciden):

1. Registrar la falla en el acta con detalle tecnico.
2. Convocar al tercer custodio (si solo se convocaron 2) para intentar con otra combinacion de fragmentos.
3. Si todas las combinaciones fallan, declarar la clave como irrecoverable y activar el Plan de Contingencia (`docs/policy/PLAN-CONTINGENCIA.md`), que incluye la generacion de una nueva CA raiz.
4. Registrar el incidente como `CeremonyStatus::Aborted`:

```
let aborted_record = ceremony.abort(
  "Recombinacion Shamir fallida. 3 combinaciones intentadas,
    todas con hash incorrecto.",
  timestamp_utc,
);
// aborted_record.status == CeremonyStatus::Aborted
```

8. Procedimiento de destruccion de clave (end-of-life)

8.1 Causales de destruccion

- 1. **Expiracion del certificado raiz** (2034-01-01) sin renovacion.
- 2. **Compromiso confirmado** de la clave privada de la CA raiz.
- 3. **Migracion completa** a una nueva CA raiz (post-cuantica u otra).
- 4. **Cese de operaciones** de la PKI de Goya Ledger.

8.2 Procedimiento

Paso	Accion	Responsable	Verificacion
8.2.1	Convocar ceremonia de destruccion con los mismos requisitos de participantes que la ceremonia de generacion.	Administrador	Notificacion formal con 10 dias habiles de anticipacion.
8.2.2	Ejecutar apertura y verificacion de entorno (pasos 5.0 y 5.1).	Administrador	Mismos requisitos.
8.2.3	Verificar que todos los certificados emitidos por la CA han sido revocados o han expirado.	Auditor	Revisar la CRL y/ o el servicio OCSP.
8.2.4	Destruir la clave privada en el HSM.	Administrador	Utilizar el comando de zeroizacion del fabricante del HSM.

```
# Destruccion de la clave en el HSM
pkcs11-tool --module "$HSM_PKCS11_LIB" --login --pin "$USER_PIN" \
  --delete-object --type privkey --label "goya-root-ca-ecdsa-
    p256"

# Verificar que la clave ya no existe
pkcs11-tool --module "$HSM_PKCS11_LIB" --login --pin "$USER_PIN" \
  --list-objects --type privkey
# No debe listar la clave eliminada

# Para destruccion completa del HSM (zeroizacion):
```

Consultar el manual del fabricante. Ejemplo SoftHSM:

softhsm2-util --delete-token --serial <numero_serie>

Paso	Accion	Responsable	Verificacion
8.2.5	Convocar a los 3 custodios para la destruccion de los fragmentos Shamir.	Administrador	Los 3 custodios deben estar presentes.
8.2.6	Cada custodio entrega su sobre sellado.	Custodios	Notario verifica los sellos y numeros de serie.
8.2.7	Destruir fisicamente los fragmentos mediante triturado de nivel P-7 (DIN 66399) o incineracion controlada.	Administrador	En presencia de todos los participantes.
8.2.8	Registrar la destruccion de cada fragmento en el acta, indicando metodo y hora.	Auditor	Fotografiar el proceso de destruccion.
8.2.9	Si existe clave PQC (ML-DSA-65), repetir los pasos 8.2.4 a 8.2.8 para dicha clave y sus fragmentos.	Administrador	Registrar ambas destruccion.
8.2.10	Publicar aviso de revocacion de la CA raiz en los repositorios correspondientes.	Administrador	Actualizar la lista de CAs de confianza.
8.2.11	Archivar permanentemente el acta de destruccion junto con las actas de generacion y recuperacion anteriores.	Auditor	Conservacion minima de 15 anos segun DS 181.
8.2.12	Cerrar la ceremonia segun el paso 5.10.	Administrador	Acta firmada y protocolizada ante notario.

9. Registro y evidencia

9.1 Acta de ceremonia

Cada ceremonia genera un acta formal que constituye el registro de auditoria primario. El acta debe contener:

Seccion del acta	Contenido
Encabezado	Tipo de ceremonia (generacion/recuperacion/destruccion), fecha y hora UTC de inicio y fin, lugar.
Participantes	Nombre completo, RUN, rol, organizacion de cada participante. Verificacion de identidad por el notario.
Configuracion de ceremonia	Umbral Shamir (2), total de fragmentos (3), notario requerido (si), minimo de testigos (2). Corresponde a los campos de CeremonyConfig.
Registro de pasos	Cada paso ejecutado con: nombre del paso (CeremonyStep), hora de ejecucion, persona responsable, notas.
Material criptografico	Fingerprint SHA-256 de la clave publica, algoritmo (key_algorithm), CN del certificado, fechas de validez, OID de politica.
Fragmentos Shamir	Numero de fragmento, numero de sello del sobre, custodio asignado. NO el contenido del fragmento.
Condiciones ambientales	Temperatura, humedad, estado de la sala, verificaciones de air-gap.
Incidentes	Cualquier desviacion del procedimiento, con descripcion, impacto y resolucion.
Firmas	Firma manuscrita de todos los participantes. Firma y sello del notario.

9.2 Registro digital (CeremonyRecord)

Ademas del acta fisica, se genera un registro digital con integridad criptografica:

```
// Estructura del registro digital (src/pki_ceremony.rs)
CeremonyRecord {
    id: String,                // Identificador unico (UUID)
    ceremony_type: String,     // "root_key_generation",
                             "intermediate_key_generation",
                             // "key_recovery", "key_destruction"
    config: CeremonyConfig {
        threshold: 2,
        total_shares: 3,
```

```

        notary_required: true,
        min_witnesses: 2,
    },
    participants: Vec<CeremonyParticipant>,
    steps: Vec<CompletedStep>,
    key_fingerprint: String, // SHA-256 de la clave publica
    key_algorithm: String,   // "ECDSA-P256" o "ML-DSA-65"
    started_at: u64,         // Timestamp UNIX UTC
    completed_at: Option<u64>,
    status: CeremonyStatus,  // Completed | Aborted
    record_hash: String,     // SHA-256 de la cadena canonica
}

// Hash de integridad: SHA-256 de
// "id|ceremony_type|fingerprint|algorithm|started_at|
//   completed_at|participants.len|steps.len"

```

La integridad del registro se verifica con `verify_record(&record)`.

9.3 Evidencia fotografica

Se tomaran fotografias de los siguientes momentos (sin capturar material criptografico en pantalla):

1. Sala antes de la ceremonia (estado inicial).
2. Verificacion de identidad de participantes (cedulas boca abajo, solo para evidenciar el acto).
3. HSM antes de la inicializacion (numero de serie visible).
4. Estacion de trabajo sin disco duro (bahia abierta visible).
5. Sobres sellados con fragmentos (numeros de sello visibles).
6. Entrega de sobres a custodios.
7. Firma del acta por todos los participantes.
8. Sala despues de la ceremonia (estado final).

Prohibicion: No se tomaran fotografias de pantallas que muestren claves, fragmentos, PINes o cualquier material criptografico sensible.

9.4 Hashes de referencia

Se registran en el acta los siguientes hashes SHA-256:

Elemento	Comando para obtener el hash
Medio de arranque USB	<code>sha256sum /dev/sdX</code> (dispositivo completo)
Binario rust-bc	<code>sha256sum /opt/goya/rust-bc</code>
Clave publica raiz (PEM)	<code>sha256sum root-ca-pubkey.pem</code>
Certificado raiz (PEM)	<code>sha256sum root-ca-cert.pem</code>
Certificado intermedio (PEM)	<code>sha256sum intermediate-ca-cert.pem</code>
Cadena de certificados (PEM)	<code>sha256sum chain.pem</code>
Registro digital (JSON)	<code>sha256sum ceremony-record.json</code>

9.5 Custodia del acta

Copia	Custodia	Plazo de conservacion
Original	Protocolo notarial (notaria publica)	Indefinido (segun legislacion notarial chilena)
Copia autorizada 1	Boveda de seguridad de la organizacion	15 anos minimo (DS 181)
Copia autorizada 2	Auditor interno	Hasta la siguiente auditoria de certificacion
Registro digital	Almacenamiento cifrado off-site	15 anos minimo

10. Anexos

Anexo A: Checklist pre-ceremonia

El administrador debe completar esta lista de verificacion al menos 48 horas antes de la ceremonia y nuevamente el dia de la ceremonia:

CHECKLIST PRE-CEREMONIA DE CLAVES -- GOYA LEDGER PKI

=====

Fecha de la ceremonia: _____
Tipo de ceremonia: ☐ Generacion raiz ☐ Generacion intermedia
 ☐ Recuperacion ☐ Destruccion

SALA SEGURA

- ☐ Sala reservada y disponible para la fecha programada
- ☐ Acceso fisico controlado verificado (cerradura funcional)
- ☐ Conectividad de red deshabilitada
- ☐ Camaras de vigilancia: grabacion local o desactivadas
- ☐ Mesa de trabajo despejada
- ☐ Impresora local disponible (no en red)
- ☐ Camara fotografica disponible (sin conexion de red)

EQUIPAMIENTO

- ☐ HSM disponible y funcional (S/N: _____)
 - ☐ Certificacion FIPS 140-3: Level ____
 - ☐ Firmware actualizado y verificado
- ☐ Estacion de trabajo air-gapped verificada (sin disco duro)
- ☐ Medio de arranque USB preparado
 - ☐ SHA-256 del medio: _____
- ☐ Binario rust-bc compilado y verificado
 - ☐ SHA-256 del binario: _____
 - ☐ Version del binario: _____
- ☐ Sobres de seguridad tamper-evident (cantidad: ____)
- ☐ Numeros de sello registrados: _____

- ☐ Medios USB cifrados para exportacion (cantidad: ____)
- ☐ Papel resistente para impresion de fragmentos

PARTICIPANTES

- ☐ Administrador de CA confirmado: _____
- ☐ Custodio 1 confirmado: _____
- ☐ Custodio 2 confirmado: _____
- ☐ Custodio 3 confirmado: _____
- ☐ Testigo 1 confirmado: _____
- ☐ Testigo 2 confirmado: _____
- ☐ Notario Publico confirmado: _____
- ☐ [] Notaria: _____
- ☐ Auditor Interno confirmado: _____
- ☐ Verificadas incompatibilidades entre participantes

DOCUMENTACION

- ☐ Este procedimiento impreso y firmado
- ☐ CP vigente disponible (OID 1.3.6.1.4.1.99999.2.1)
- ☐ CPS vigente disponible (OID 1.3.6.1.4.1.99999.2.2)
- ☐ Plan de Seguridad disponible
- ☐ Plan de Contingencia disponible
- ☐ Plantillas de acta impresas (cantidad: ____)
- ☐ Plantillas de sobre de custodio impresas (cantidad: ____)
- ☐ Lista de participantes impresa

SOFTWARE (verificar en la estacion air-gapped)

- ☐ OpenSSL >= 3.0.0 instalado
- ☐ pkcs11-tool (OpenSC) instalado
- ☐ Biblioteca PKCS#11 del HSM disponible
- ☐ [] Ruta: _____
- ☐ Rust nightly toolchain instalado
- ☐ cargo disponible

Verificado por: _____ Fecha: _____
Firma: _____

Anexo B: Plantilla de acta de ceremonia

=====

ACTA DE CEREMONIA DE CLAVES

INFRAESTRUCTURA DE CLAVE PUBLICA -- GOYA LEDGER

=====

ACTA N.o: _____

TIPO DE CEREMONIA: ☐ Generacion de clave raiz
☐ Generacion de CA intermedia
☐ Recuperacion de clave
☐ Destruccion de clave

LUGAR: _____

FECHA: _____ HORA INICIO (UTC): _____

HORA FIN (UTC): _____

SECCION 1: PARTICIPANTES

Todos los participantes han sido identificados mediante cedula de identidad chilena vigente por el notario publico firmante.

1. Administrador de CA

Nombre: _____

RUN: _____

Organizacion: _____

2. Custodio 1

Nombre: _____

RUN: _____

Organizacion: _____

3. Custodio 2

Nombre: _____

RUN: _____

Organizacion: _____

4. Custodio 3

Nombre: _____

RUN: _____

Organizacion: _____

5. Testigo 1

Nombre: _____

RUN: _____

Organizacion: _____

6. Testigo 2

Nombre: _____

RUN: _____

Organizacion: _____

7. Notario Publico

Nombre: _____

Notaria: _____

Jurisdiccion: _____

8. Auditor Interno

Nombre: _____

RUN: _____

Organizacion: _____

SECCION 2: CONFIGURACION DE CEREMONIA

Esquema Shamir:

Umbral (threshold): 2

Total de fragmentos (total_shares): 3

Notario requerido: Si
Minimo de testigos: 2

SECCION 3: VERIFICACION DE ENTORNO

HSM:

Fabricante: _____
Modelo: _____
Numero de serie: _____
Certificacion FIPS 140-3: Level ____
Estado inicial: [] Fabrica [] Particion vacia

Estacion de trabajo:

Sin disco duro: [] Verificado por testigos
Medio de arranque USB SHA-256: _____
Binario rust-bc SHA-256: _____
Conectividad de red: [] Deshabilitada verificada

Condiciones ambientales:

Temperatura: _____ C
Humedad relativa: _____ %

SECCION 4: REGISTRO DE PASOS

N.o	Paso	Hora (UTC)	Responsable	Resultado	Notas

(Agregar filas segun sea necesario)

SECCION 5: MATERIAL CRIPTOGRAFICO

Clave primaria (ECDSA P-256):

Fingerprint SHA-256: _____
Etiqueta HSM: goya-root-ca-ecdsa-p256
Algoritmo: ECDSA P-256

Clave PQC (ML-DSA-65) (si aplica):

Fingerprint SHA-256: _____
Etiqueta HSM: goya-root-ca-ml-dsa-65
Algoritmo: ML-DSA-65

Certificado raiz:

CN: Rust-BC Internal CA
Validez: 2024-01-01 a 2034-01-01
Fingerprint SHA-256 del certificado: _____
OID de politica: 1.3.6.1.4.1.99999.2.1
URI de CPS: https://goya.cl/pki/cp

Certificado intermedio (si aplica en esta ceremonia):
CN: Goya Ledger Intermediate CA
Validez: _____ a _____
Fingerprint SHA-256 del certificado: _____

SECCION 6: FRAGMENTOS SHAMIR

Fragmento	Sello N.o	Custodio	Firma custodio
1 de 3			
2 de 3			
3 de 3			

Verificacion de recombinacon:
{S1,S2}: [] Exitosa [] Fallida
{S1,S3}: [] Exitosa [] Fallida
{S2,S3}: [] Exitosa [] Fallida

SECCION 7: INCIDENTES Y DESVIACIONES

[] Ninguno
[] Se registran los siguientes incidentes:

SECCION 8: HASHES DE REFERENCIA

Elemento	SHA-256
Medio de arranque USB	
Binario rust-bc	
Clave publica raiz (PEM)	
Certificado raiz (PEM)	
Cert. intermedio (PEM)	
Cadena de certs. (PEM)	
Registro digital (JSON)	

SECCION 9: RESULTADO

La ceremonia se declara: [] COMPLETADA [] ABORTADA

Motivo de aborto (si aplica): _____

Hash del registro digital (record_hash): _____

=====

SECCION 10: FIRMAS

=====

Administrador de CA:

Firma: _____ Fecha: _____

Custodio 1:

Firma: _____ Fecha: _____

Custodio 2:

Firma: _____ Fecha: _____

Custodio 3:

Firma: _____ Fecha: _____

Testigo 1:

Firma: _____ Fecha: _____

Testigo 2:

Firma: _____ Fecha: _____

Auditor Interno:

Firma: _____ Fecha: _____

Notario Publico:

Firma: _____ Fecha: _____

Sello: [SELLO NOTARIAL]

=====

FIN DEL ACTA

=====

Anexo C: Plantilla de sobre de custodio

=====

DECLARACION DE CUSTODIA DE FRAGMENTO DE CLAVE
INFRAESTRUCTURA DE CLAVE PUBLICA -- GOYA LEDGER

=====

FRAGMENTO N.o: ____ de ____

SELLO DEL SOBRE N.o: _____

FECHA DE ENTREGA: _____

CEREMONIA ACTA N.o: _____

FINGERPRINT DE LA CLAVE:

DATOS DEL CUSTODIO:

Nombre completo:

RUN: _____

Organizacion:

Direccion de contacto:

Telefono de contacto:

Correo electronico:

OBLIGACIONES DEL CUSTODIO

El custodio firmante declara conocer y aceptar las siguientes obligaciones:

1. Almacenar el sobre sellado en caja fuerte, boveda bancaria u otro medio de almacenamiento seguro con acceso restringido.
2. No abrir el sobre bajo ninguna circunstancia, excepto durante una ceremonia formal de recuperacion de clave convocada por el Administrador de la CA conforme al procedimiento `PROCEDIMIENTO-CEREMONIA-CLAVES.md`, Seccion 7.
3. Verificar periodicamente (al menos cada 90 dias) que el sello del sobre se encuentra intacto.
4. Reportar inmediatamente al Administrador de la CA cualquier:
 - a) Dano o deterioro del sobre o su sello.
 - b) Intento de acceso no autorizado al sobre.
 - c) Perdida o robo del sobre.
 - d) Cualquier circunstancia que pueda comprometer la seguridad del fragmento.
5. Presentarse fisicamente con el sobre cuando sea convocado para una ceremonia de recuperacion, con un plazo maximo de respuesta de 48 horas desde la convocatoria.
6. No delegar la custodia del sobre a terceros sin autorizacion escrita del Administrador de la CA.
7. En caso de renuncia, despido o imposibilidad de continuar con la custodia, devolver el sobre al Administrador de la CA para su destruccion y reemision en nueva ceremonia.
8. Mantener absoluta confidencialidad sobre su condicion de custodio y sobre cualquier detalle del contenido del sobre.

FIRMA

Declaro haber recibido el sobre sellado identificado arriba y
acepto las obligaciones descritas en este documento.

Custodio:

Firma: _____

Nombre: _____

Fecha: _____

Administrador de CA (entrega):

Firma: _____

Nombre: _____

Fecha: _____

Notario Publico (fe de entrega):

Firma: _____

Sello: [SELLO NOTARIAL]

=====

Anexo D: Procedimiento de aborto de ceremonia

En cualquier momento de la ceremonia, el administrador, el auditor o el notario
pueden declarar el aborto de la ceremonia si:

1. Se detecta una violacion de seguridad (acceso no autorizado, dispositivo
electronico no declarado, conectividad de red detectada).
2. El HSM presenta un fallo de hardware.
3. Un participante obligatorio debe abandonar la sala antes de completar la
ceremonia.
4. Se detecta una discrepancia en los hashes de verificacion.
5. Cualquier otra circunstancia que comprometa la integridad del procedimiento.

Pasos de aborto:

Paso	Accion	Responsable
D.1	Declarar el aborto en voz alta, indicando la causal.	Administrador, auditor o notario
D.2	Detener inmediatamente toda operacion criptografica.	Administrador
D.3	Si se habian generado claves, destruirlas del HSM (pkcs11-tool -- delete-object).	Administrador
D.4	Si se habian impreso fragmentos Shamir, destruirlos fisicamente.	Administrador

Paso	Accion	Responsable
D.5	Registrar el aborto en el acta con la causal, hora y persona que lo declaro.	Auditor
D.6	Cerrar la sala y ejecutar el paso 5.10.	Administrador
D.7	Programar nueva ceremonia una vez resuelta la causal del aborto.	Administrador

```

let aborted_record = ceremony.abort(
  "Causal: [descripcion]. Declarado por: [nombre]. Hora:
    [UTC].",
  timestamp_utc,
);
// aborted_record.status == CeremonyStatus::Aborted

```

Anexo E: Correspondencia con la implementacion en codigo

Seccion del procedimiento	Archivo fuente	Estructura/Funcion
Roles (Seccion 3)	src/pki_ceremony.rs	CeremonyRole enum
Pasos (Seccion 5)	src/pki_ceremony.rs	CeremonyStep enum
Configuracion Shamir	src/pki_ceremony.rs	CeremonyConfig::default()
Registro de ceremonia	src/pki_ceremony.rs	CeremonyRecord struct
Validacion de ceremonia	src/pki_ceremony.rs	KeyCeremony::validate()
Hash de integridad	src/pki_ceremony.rs	compute_record_hash()
Verificacion de registro	src/pki_ceremony.rs	verify_record()
Generacion CA raiz	src/pki.rs	NodeCaConfig::generate()
Jerarquia CA	src/pki.rs	CaHierarchy::generate()
Emision de certificados	src/pki.rs	sign_node_cert()
Extensiones X.509	src/pki.rs	certificate_policies_extension()
QC Statements	src/pki.rs	qc_statements_extension()
Integracion HSM	src/identity/hsm.rs	HsmSigningProvider HsmConfig::from_env()

Seccion del procedimiento	Archivo fuente	Estructura/Funcion
Configuracion HSM	src/identity/hsm.rs	
Variables de entorno HSM	src/identity/hsm.rs	HSM_PKCS11_LIB, HSM_SLOT_ID, HSM_PIN, HSM_KEY_LABEL
Respaldo de claves	src/identity/hsm.rs	HsmSigningProvider::backup_info()
Constantes CA	src/pki.rs	INTERNAL_CA_CN, INTERMEDIATE_CA_CN, CA_NOT_BEFORE, CA_NOT_AFTER
OIDs de politica	src/pki_policy.rs	CP_OID, CPS_OID, GOYA_OID_ROOT

Fin del documento.

Este procedimiento debe revisarse y actualizarse al menos una vez al ano o cuando se produzcan cambios en la normativa aplicable, en la infraestructura de hardware, o en los algoritmos criptograficos utilizados por Goya Ledger.